作业五

一、 CUDA 程序中常见的内存类型包括 全局内存、共享内存、寄存器、本地内存、常量内存等。请结合下面代码，回答以下问题：

```
__constant__ float c_alpha;

__device__ float d_total;

__global__ void memoryDemo(float *g_in, float *g_out, int n) {
    __shared__ float s_tile[256];

    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    float temp = 0.0f;
    float localArray[4];

    if (idx < n) {
        s_tile[tid] = g_in[idx];

        __syncthreads();

        temp = s_tile[tid] * c_alpha;
        localArray[0] = temp;

        g_out[idx] = localArray[0];

        atomicAdd(&d_total, temp);
    }
}
```

1. s_tile 属于哪类内存类型，简要描述该类内存的访问范围与生命周期
2. c_alpha 属于哪类内存？适合存放什么数据？

二、 阅读下面 CUDA 程序片段，回答以下问题：

```
__global__ void vecAdd(float *a, float *b, float *c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    c[i] = a[i] + b[i];
}

int main() {
    int n = 1000;
    size_t size = n * sizeof(float);

    float *d_a, *d_b, *d_c;

    cudaMalloc((void**)&d_a, size);
    cudaMalloc((void**)&d_b, size);
    cudaMalloc((void**)&d_c, size);

    int threads = 256;
    int blocks = (n + threads - 1) / threads;

    vecAdd<<<blocks, threads>>>(d_a, d_b, d_c, n);
}
```

1. 这段核函数存在什么潜在的错误？
2. 当 n = 1000 时，一共启动多少个线程？是否可能发生对数组的越界访问？

三、 假设有一个 N × N 的矩阵 M 存储在一维数组 float* mat 中（按行优先存储），要在 GPU 上做矩阵转置，kernel 如下：

```
__global__ void Transpose(float* mat, float* matT, int N) {
        int row = blockIdx.y * blockDim.y + threadIdx.y;
        int col = blockIdx.x * blockDim.x + threadIdx.x;

        if (row < N && col < N) {
            matT[col * N + row] = mat[row * N + col];
        }
}
```

主机端启动方式：

```
dim3 blockSize(16, 16);
dim3 gridSize( (N + blockSize.x - 1)/blockSize.x,
                (N + blockSize.y - 1)/blockSize.y );

Transpose<<<gridSize, blockSize>>>(mat, matT, N);
```

请回答：

1. 如果 N = 1000，计算 gridSize.x 和 gridSize.y 各是多少？

2. 假设 GPU 有 1024 个线程每个 SM 最多可以同时执行，blockSize 设为 (16,16) 后，每个 block 占用多少个线程？最后一个 block 中可能有多少线程 不执行计算？

3. 如果去掉 kernel 中的边界判断 if(row<N && col<N)，会发生什么问题？为什么这个判断很重要？

四、 假设有一个 N×N 的矩阵 M 存储在一维数组 float* mat 中，按行优先方式存储。现在希望在 GPU 上实现“把矩阵中每个元素都乘以 2”的操作，kernel 框架如下：

```
__global__ void MatScale2(float* mat, int N) {
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;

if (_____) {
    mat[_____] = 2.0f * mat[_____];
}

}
```

主机端启动方式如下：

```
dim3 blockSize(16, 16);
dim3 gridSize((N + blockSize.x - 1) / blockSize.x,
(N + blockSize.y - 1) / blockSize.y);

MatScale2<<<gridSize, blockSize>>>(mat, N);
```

请完成下列问题：

1. 如果某线程满足 row=3，col=5，那么它处理的是矩阵中的哪一个元素？
2. 补全上面的 kernel，使每个线程对自己负责的矩阵元素乘以 2
3. 若 N=1000，计算 gridSize.x 和 gridSize.y 的值，并说明为什么 kernel 中必须写边界判断